Geavanceerde Computerarchitectuur Labo 5
Coalesced memory access and device (GPU) memory types

Arno De Lathauwer
Indra Coppens

Maart 2025

1 Context

In dit labo onderzoeken we de invloed van synchrone en asynchrone processing op de uitvoeringstijd van een CUDA-kernel. Dit doen we door een kernel te maken die vier verschillende operaties uitvoert: de maximale en minimale waarde in een array bepalen en de som en het product van alle elementen in een array bepalen. Vervolgens wordt deze kernel op een synchrone en asynchrone manier uitgevoerd en worden de uitvoeringstijden voor beide processing types vergeleken.

2 Methodologie

Voor de kernel bouwen we verder op de kernel uit labo 2 die de maximale waarde in een array bepaald. Door een bepaalde integer waarde mee te geven (`int op`), weet de kernel welke van de vier operaties er uitgevoerd moet worden op de array. Eens de kernel aangepast was om alle types operaties te ondersteunen, werd nagegaan of de kernel correct werkte. Dit bleek het geval te zijn.

Vervolgens werden twee methodes gedefinieerd die de kernel op een synchrone en een asynchrone manier aanroepen.

2.1 Synchrone processing

Bij synchrone processing wordt er gebruik gemaakt van de functie call `cudaMemcpy` om data van de CPU naar de GPU te kopiëren. Dit is een synchrone operatie [2], wat inhoudt dat de CPU wacht tot de operatie volledig voltooid is alvorens er verder gegaan wordt met uitvoeren. Verder roepen we de CUDA-kernel op met de default stream (= `cudaStreamDefault`). Hierdoor wordt impliciet synchronisatie afgedwongen [3]. Daarnaast wordt gebruik gemaakt van de functie `cudaDeviceSynchronize()`. Deze functie zorgt ervoor dat de CPU wacht tot alle bewerkingen op de GPU voltooid zijn alvorens er verder gegaan wordt [4].

Wanneer er gebruik gemaakt wordt van synchrone processing zal er een array worden aangemaakt en deze zal gevuld worden met waarden. Daarna wordt deze array gekopieerd van de CPU naar de GPU, de CPU wacht tot de array volledig gekopieerd is. Vervolgens wordt de CUDA-kernel opgeroepen met de correcte `op`-parameter en wordt de geselecteerde operatie uitgevoerd. De CPU voert de `cudaDeviceSynchronize()` functie uit en wacht tot de operatie op de GPU voltooid is alvorens er een nieuwe array wordt aangemaakt, gevuld en een nieuwe kernel wordt gestart. De uitvoering van het programma ziet eruit zoals op *figuur 1.*

2.2 Asynchrone processing

Bij asynchrone processing wordt er gebruik gemaakt van de functie call `cudaMemcpyAsync` om data van de CPU naar de GPU te kopiëren. Hier hebben we te maken met een synchrone operatie [2], wat inhoud dat de CPU niet wacht tot de kopieer operatie voltooid is en er dus onmiddelijk verder gegaan wordt met uitvoeren. Daarnaast maken we gebruik van streams zodat operaties die deel uitmaken van dezelfde stream in de juiste volgorde worden uitgevoerd en zodat er effectief asynchrone processing optreedt.

Bij asynchrone processing zal er een array (horende bij bijvoorbeeld `stream1`) aangemaakt en gevuld worden. Vervolgens wordt deze array van de CPU naar de GPU gekopieerd. Echter wacht de CPU nu niet tot deze kopieeroperatie is uitgevoerd en er wordt onmiddelijk verder gegaan met het uitvoeren. De kernel (horende bij `stream1`) wordt opgeroepen van zodra de bijhorende array gekopieerd is. Er wordt bij asynchrone processing geen gebruik gemaakt van de `cudaDeviceSynchronize()` functie, waardoor de CPU tijdens het uitvoeren van de kernel (horende

bij `stream1`) de volgende array, voor `stream2` al kan aanmaken en vullen met waarden. De uitvoering van het programma ziet eruit zoals op *figuur 2*.

3 Observaties

Bij de synchrone uitvoering gaat de CPU pas verder wanneer de GPU de verwerking voltooid heeft. Dit resulteert in een vertraging, aangezien elke operatie op de GPU volledig voltooid moet zijn voordat de CPU verder kan gaan met de volgende stap.

In het geval van asynchrone verwerking wordt de CPU niet geblokkeerd en kan deze doorgaan met andere taken (zoals hier: het voorbereiden van de data voor de volgende GPU-oproep) terwijl de GPU bezig is. Dit kan leiden tot snellere algemene doorlooptijden omdat de CPU zijn tijd optimaal benut, zelfs als de GPU bezig is.

Met behulp van de chrono library meten we de tijd die nodig is om 100 iteraties uit te voeren van eenzelfde programma (zie `Figuur 3`). Dit programma omvat de 4 operaties (max, min, som en product) die op de GPU worden uitgevoerd met daartussen telkens het aanmaken van een nieuwe array op de CPU. De bijhorende code is te vinden in `Appendix A`. In onze metingen bleek dat de synchrone verwerking voor bijna alle array groottes een snellere uitvoeringstijd had dan de asynchrone werking. Dit is niet in lijn met onze verwachtingen. We verwachten dat de asynchrone werking sneller zou moeten zijn dan de synchrone werking. Enkel bij een array grootte van 2 zien we een verschil die enigszins aan onze verwachtingen voldoet. We besluiten dus dat er ergens een fout zit in de uitvoering van het programma wat deze incorrecte meetresultaten oplevert. We hebben geprobeerd deze fout te vinden, met als resultaat dat we enkele kleine fouten vonden en deze opgelost hebben. De resultaten bleven echter hetzelfde als voorheen.

4 Conclusie

We concluderen dat de asynchrone uitvoering een betere efficiëntie biedt dan de synchrone verwerking, omdat de CPU niet hoeft te wachten op de GPU en tegelijkertijd andere taken kan uitvoeren. Dit verkort de totale doorlooptijd van de toepassing. De synchrone verwerking daarentegen leidt tot vertraging, omdat de CPU wacht tot de GPU elke taak volledig heeft voltooid. Onze metingen bevestigen deze conclusie echter niet. Dit wijst erop dat er ergens een fout in de geschreven code zit.

5 Graphs and figures

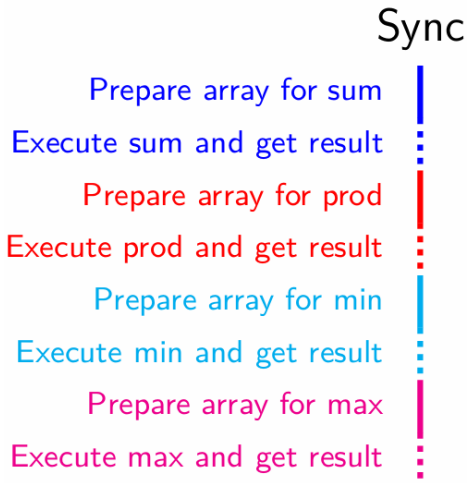


Figure 1: Synchronous processing

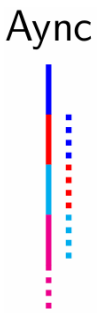


Figure 2: Asynchronous processing

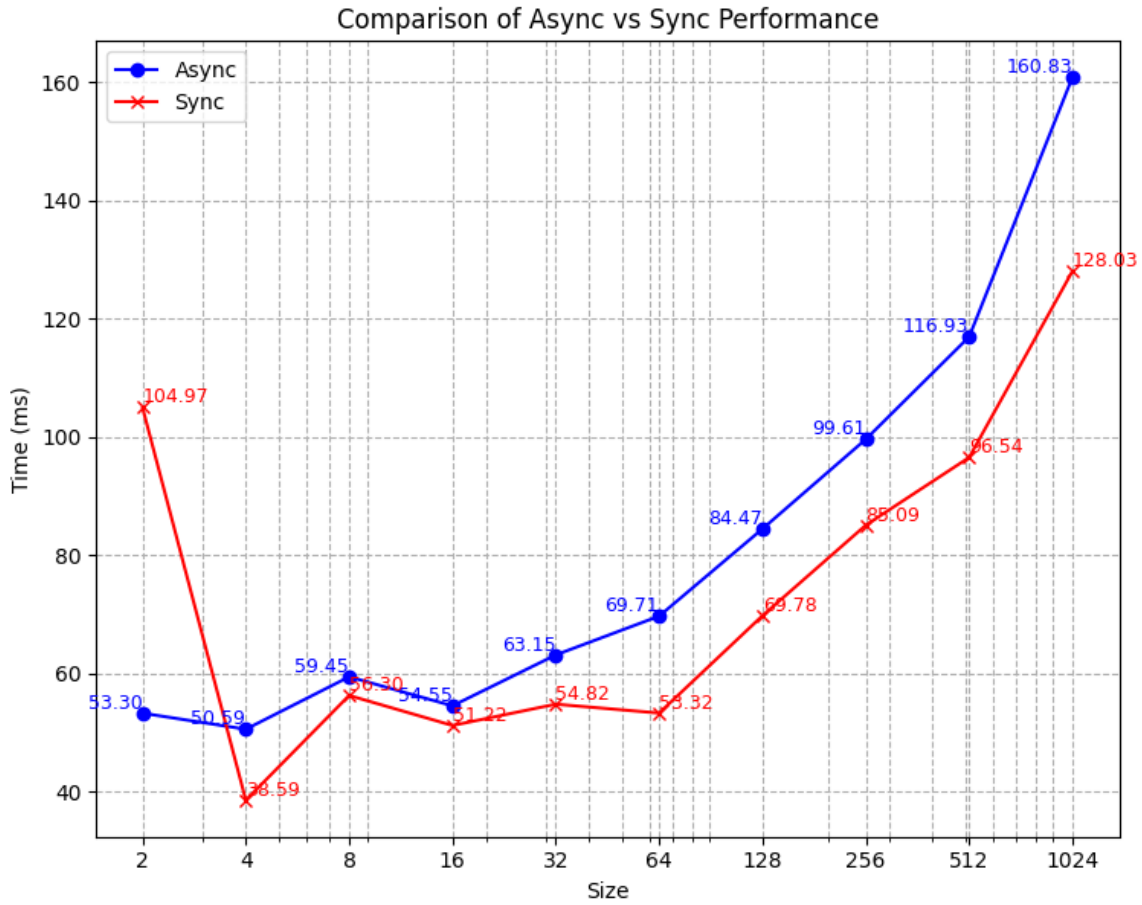


Figure 3: Uitvoeringstijd van 100 iteraties met de 4 GPU-operaties (async vs sync)

Appendix A : Code

```
#include <iostream>
#include <fstream>
#include <vector>
#include <cuda_runtime.h>
#include <chrono>

__global__ void ComputeArray(int *d_arr, int log_2_size, int select)
{
    int idx = threadIdx.x + blockIdx.x * blockDim.x;

    for (int iteratie = 1; iteratie <= log_2_size; iteratie++)
    {
        if (idx < pow(2, log_2_size) / (pow(2, iteratie)))
        {
            int val1 = idx * pow(2, iteratie);
            int val2 = idx * pow(2, iteratie) + (pow(2, iteratie) / 2);

            if (select == 0)
            { // max // op0
                if (d_arr[val2] > d_arr[val1])
                {
                    d_arr[val1] = d_arr[val2];
                }
            }
            else if (select == 1)
            { // min // op1
                if (d_arr[val2] < d_arr[val1])
                {
                    d_arr[val1] = d_arr[val2];
                }
            }
            else if (select == 2)
            { // sum // op2
                d_arr[val1] += d_arr[val2];
            }
            else
            { // prod // op3
                d_arr[val1] *= d_arr[val2];
            }

            __syncthreads();
        }
    }
}

void sync_mode()
{
    std::vector<float> time;
    std::vector<int> size;
    for (int i = 2; i < 1025; i = i * 2)
    {

        int N = i;
        size.push_back(N);

        const auto start = std::chrono::steady_clock::now();
        for (int j = 0; j < 100; j++)
        {

            int h_arr0[N];
            int *d_arr0; // Device-array 1 (GPU)
            int h_arr1[N];
            int *d_arr1; // Device-array 2 (GPU)

            int h_arr2[N];
            int *d_arr2; // Device-array 3 (GPU)
            int h_arr3[N];
            int *d_arr3; // Device-array 4 (GPU)

            cudaMalloc((void **)&d_arr0, N * sizeof(int));
            cudaMalloc((void **)&d_arr1, N * sizeof(int));
            cudaMalloc((void **)&d_arr2, N * sizeof(int));
            cudaMalloc((void **)&d_arr3, N * sizeof(int));

            // op0
```

```
            for (int e = 0; e < N; e++)
            { // filling array
                h_arr0[e] = e;
            }

            cudaMemcpy(d_arr0, h_arr0, N * sizeof(int), cudaMemcpyHostToDevice);

            ComputeArray<<<1, N>>>(d_arr0, log2(N), 0);
            cudaDeviceSynchronize();

            cudaMemcpy(h_arr0, d_arr0, N * sizeof(int), cudaMemcpyDeviceToHost);

            //  Opruimen
            cudaFree(d_arr0);

            // op1

            for (int e = 0; e < N; e++)
            { // filling array
                h_arr1[e] = e;
            }

            cudaMemcpy(d_arr1, h_arr1, N * sizeof(int), cudaMemcpyHostToDevice);

            ComputeArray<<<1, N>>>(d_arr1, log2(N), 1);
            cudaDeviceSynchronize();

            cudaMemcpy(h_arr1, d_arr1, N * sizeof(int), cudaMemcpyDeviceToHost);

            // Opruimen
            cudaFree(d_arr1);

            // op2

            for (int e = 0; e < N; e++)
            { // filling array
                h_arr2[e] = e;
            }

            cudaMemcpy(d_arr2, h_arr2, N * sizeof(int), cudaMemcpyHostToDevice);

            ComputeArray<<<1, N>>>(d_arr2, log2(N), 1);
            cudaDeviceSynchronize();

            cudaMemcpy(h_arr2, d_arr2, N * sizeof(int), cudaMemcpyDeviceToHost);

            // Opruimen
            cudaFree(d_arr2);

            // op3

            for (int e = 0; e < N; e++)
            { // filling array
                h_arr3[e] = e;
            }

            cudaMemcpy(d_arr3, h_arr3, N * sizeof(int), cudaMemcpyHostToDevice);

            ComputeArray<<<1, N>>>(d_arr3, log2(N), 1);
            cudaDeviceSynchronize();

            cudaMemcpy(h_arr3, d_arr3, N * sizeof(int), cudaMemcpyDeviceToHost);

            // Opruimen
            cudaFree(d_arr3);
        }
        cudaDeviceSynchronize();
        const auto end = std::chrono::steady_clock::now();

        const std::chrono::duration<double> elapsed_seconds{end - start};
        float milliseconds = elapsed_seconds.count() * 1000;
        time.push_back(milliseconds);
        std::cout << elapsed_seconds.count() << "\n";
    }
    // Schrijf naar een bestand
    std::ofstream outFile("sync.txt");
    if (!outFile)
```

```
    {
        std::cerr << "Kon bestand niet openen!" << std::endl;
    }

    for (size_t i = 0; i < time.size(); i++)
    {
        outFile << time[i] << " " << size[i] << "\n";
    }

    outFile.close();
    std::cout << "Data opgeslagen in sync.txt\n";
}

void async_mode()
{
    std::vector<float> time;
    std::vector<int> size;

    for (int i = 2; i < 1025; i = i * 2)

    {
        int N = i;
        size.push_back(N);
        const auto start = std::chrono::steady_clock::now();
        for (int j = 0; j < 100; j++)
        {

            int h_arr0[N];
            int *d_arr0; // Device-array 1 (GPU)
            int h_arr1[N];
            int *d_arr1; // Device-array 2 (GPU)

            int h_arr2[N];
            int *d_arr2; // Device-array 3 (GPU)
            int h_arr3[N];
            int *d_arr3; // Device-array 4 (GPU)

            cudaMalloc((void **)&d_arr0, N * sizeof(int));
            cudaMalloc((void **)&d_arr1, N * sizeof(int));
            cudaMalloc((void **)&d_arr2, N * sizeof(int));
            cudaMalloc((void **)&d_arr3, N * sizeof(int));

            cudaStream_t stream0;
            cudaStreamCreate(&stream0);
            cudaStream_t stream1;
            cudaStreamCreate(&stream1);
            cudaStream_t stream2;
            cudaStreamCreate(&stream2);
            cudaStream_t stream3;
            cudaStreamCreate(&stream3);

            // op0
            for (int e = 0; e < N; e++)
            { // filling array
                h_arr0[e] = e;
            }

            cudaMemcpyAsync(d_arr0, h_arr0, N * sizeof(int), cudaMemcpyHostToDevice, stream0);

            ComputeArray<<<1, N, 0, stream0>>>(d_arr0, log2(N), 0);

            cudaMemcpyAsync(h_arr0, d_arr0, N * sizeof(int), cudaMemcpyDeviceToHost, stream0);

            // op1
            for (int e = 0; e < N; e++)
            { // filling array
                h_arr1[e] = e;
            }

            cudaMemcpyAsync(d_arr1, h_arr1, N * sizeof(int), cudaMemcpyHostToDevice, stream1);

            ComputeArray<<<1, N, 0, stream1>>>(d_arr1, log2(N), 1);

            cudaMemcpyAsync(h_arr1, d_arr1, N * sizeof(int), cudaMemcpyDeviceToHost, stream1);

            // op2
            for (int e = 0; e < N; e++)
            { // filling array
```

```
                h_arr2[e] = e;
            }

            cudaMemcpyAsync(d_arr2, h_arr2, N * sizeof(int), cudaMemcpyHostToDevice, stream2);

            ComputeArray<<<1, N, 0, stream2>>>(d_arr2, log2(N), 1);

            cudaMemcpyAsync(h_arr2, d_arr2, N * sizeof(int), cudaMemcpyDeviceToHost, stream2);

            // op3

            for (int e = 0; e < N; e++)
            { // filling array
                h_arr3[e] = e;
            }

            cudaMemcpyAsync(d_arr3, h_arr3, N * sizeof(int), cudaMemcpyHostToDevice, stream3);

            ComputeArray<<<1, N, 0, stream3>>>(d_arr3, log2(N), 1);

            cudaMemcpyAsync(h_arr3, d_arr3, N * sizeof(int), cudaMemcpyDeviceToHost, stream3);

            // Opruimen
            cudaFree(d_arr0);
            cudaFree(d_arr1);
            cudaFree(d_arr2);
            cudaFree(d_arr3);
        }
        cudaDeviceSynchronize();
        const auto end = std::chrono::steady_clock::now();

        const std::chrono::duration<double> elapsed_seconds{end - start};
        float milliseconds = elapsed_seconds.count() * 1000;
        time.push_back(milliseconds);
        std::cout << elapsed_seconds.count() << "\n";
    }
    // Schrijf naar een bestand
    std::ofstream outFile("async.txt");
    if (!outFile)
    {
        std::cerr << "Kon bestand niet openen!" << std::endl;
    }

    for (size_t i = 0; i < time.size(); i++)
    {
        outFile << time[i] << " " << size[i] << "\n";
    }

    outFile.close();
    std::cout << "Data opgeslagen in async.txt\n";
}

int main()
{
    sync_mode();
    async_mode();
    return 0;
}
```

References

[1] 'CUDA C++ Programming Guide'. Accessed: Mar. 10, 2025. [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html

[2] 'API synchronization behavior'. Accessed: Mar. 12, 2025. [Online]. Available: https://docs.nvidia.com/cuda/cuda-runtime-api/api-sync-behavior.html#api-sync-behavior

[3] 'CUDA C/C++ Streams and Concurrency'. Accessed: Mar. 12, 2025. [Online]. Available: https://developer.download.nvidia.com/CUDA/training/StreamsAndConcurrencyWebinar.pdf

[4] 'CUDA Runtime API'. Accessed: Mar. 12, 2025. [Online]. Available: https://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART__DEVICE.html#group__CUDART__DEVICE_1g10e20b05a95f638a4071a655503df25d